

CS 736

Medical Image Computing (MIC)

Bayesian Image Reconstruction

Suyash P. Awate

Image Reconstruction

- Optimization problem

$$P(\text{PhysicalEntityImage}|\text{Data}) = \frac{P(\text{Data}|\text{PhysicalEntityImage})P(\text{PhysicalEntityImage})}{P(\text{Data})}$$

- In CT, physical-entity image is image of attenuation coefficients
- In MRI, physical-entity image is image of magnetization vectors
- Prior model on PhysicalEntityImage (say, X)
 - Same as that for denoising, e.g., MRF or dictionary
- **Likelihood** PDF given data (say, Y)
 - Probability of generating data given physical-entity image
 - **Transformation** model (e.g., system matrix underlying CT, MRI) & **noise** model
 - If we assume additive noise:
Model $Y_{\text{vec}} := \mathbf{G} X_{\text{vec}} + \text{Noise}_{\text{vec}}$, where $\mathbf{G}$ is system **matrix**, or
Model $Y := \mathbf{G} X + \text{Noise}$, where $\mathbf{G}$ is equivalent **operator**

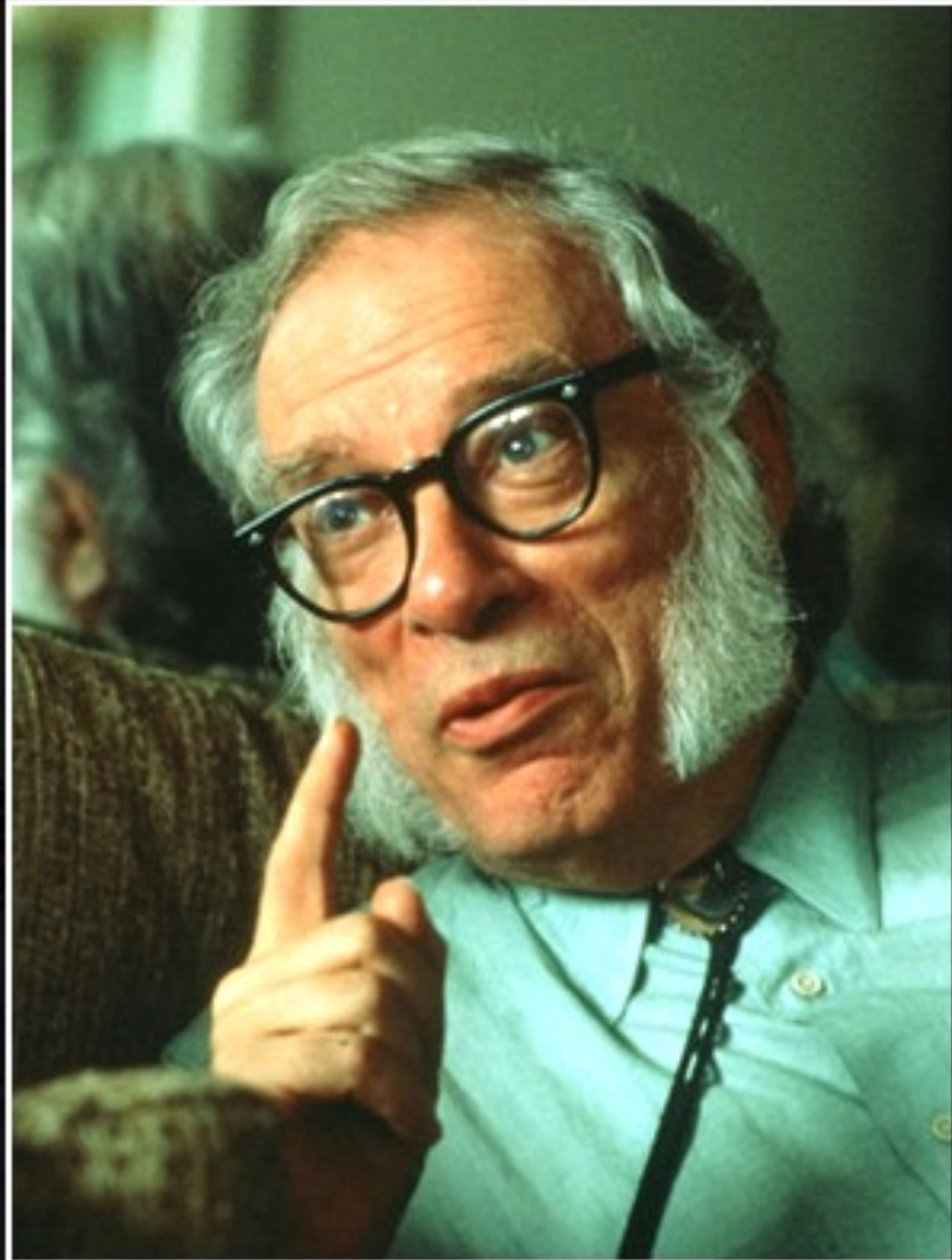
Image Reconstruction

- Optimization using gradient descent
- At chosen pixel i , derivative for log prior is:

$$\frac{\partial P(x)}{\partial x_i} = \frac{\partial}{\partial x_i} \sum_{a \in A_i} V_a(x_a) \text{ where } A \text{ is the set of cliques that contain site } i$$

- Derivative for likelihood-based term = ?
- Some function of Gx and y
- We have $y := Gx + \text{Noise}$, where x and y are column vectors
- Assuming noise is (zero-mean) i.i.d. Gaussian leads to loss function of form $\| Gx - y \|_2^2$

Mathematical Optimization is Part Art & Part Science



There is an art to science, and a science in art; the two are not enemies, but different aspects of the whole.

— *Isaac Asimov* —

AZ QUOTES

Matrix Derivatives

- Consider a function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$
- $f(\cdot)$ is a function of n variables $x := [x_1, \dots, x_n]^T$
- Function value has m components: $f_1(x_1, \dots, x_n), \dots, f_m(x_1, \dots, x_n)$
- Represent $f(x)$ as a column vector: $[f_1(x_1, \dots, x_n), \dots, f_m(x_1, \dots, x_n)]^T$
- Derivative $\partial/\partial x f(x) = \text{Jacobian matrix} = D$

$$D_{ij} := \frac{\partial f_i(\cdot)}{\partial x_j}$$

- Number of rows = number of components m
- Number of columns = number of variables n

Matrix Derivatives

- Multivariate Gradient-Descent Optimization
- Let $x = [x_1, \dots, x_n]^T$ = column vector of variables
- Let $f(x)$ = **real-valued function** of x
- Derivative $\partial/\partial x f(x)$ = is a row vector $g(x) = \left[\dots, \frac{\partial f(x)}{\partial x_i}, \dots \right]$
- For special case of real-valued functions,
transpose of derivative (derivative represented as row vector)
is called **gradient** (gradient represented as column vector)
- For gradient-descent optimization,
gradient of $f(x)$ w.r.t. x (i.e., $\nabla_x f(x)$)
needs to be of same dimension as x

Matrix Derivatives

- en.wikipedia.org/wiki/Matrix_calculus
- Using scalars, vectors, matrices
 - Total 9 possibilities
 - 6 of them can be written nicely in scalar/vector/matrix form

Types of matrix derivative

Types	Scalar	Vector	Matrix
Scalar	$\frac{\partial y}{\partial x}$	$\frac{\partial \mathbf{y}}{\partial x}$	$\frac{\partial \mathbf{Y}}{\partial x}$
Vector	$\frac{\partial y}{\partial \mathbf{x}}$	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$	
Matrix	$\frac{\partial y}{\partial \mathbf{X}}$		

Matrix Calculus

www.ee.ic.ac.uk/hp/staff/dmb/matrix/calculus.html

Go to: [Introduction](#), [Notation](#), [Index](#)

Contents of Calculus Section

- [Notation](#)
- Differentials of [Linear](#), [Quadratic](#) and [Cubic](#) Products
- Differentials of [Inverses](#), [Trace](#) and [Determinant](#)
- [Hessian](#) matrices

Matrix Derivatives

- What is gradient in our case of $\| Gx - y \|_2^2$?
- Let real-valued $f(x) := (Gx - y)^T (Gx - y)$
 - G is a matrix, x and y are column vectors
- Chain rule for derivatives
- Let $w := Gx - y$, $f(w) := w^T w$
- Then
$$\frac{\partial f(x)}{\partial x} = \left(\frac{\partial}{\partial w} f(w) \right) \left(\frac{\partial}{\partial x} Gx - y \right)$$
- $\partial/\partial w \quad f(w) \quad = 2w^T \quad [\text{row vector}]$
- $\partial/\partial x \quad (Gx-y) \quad = G \quad [\text{matrix}]$
- $\partial/\partial x \quad f(x) \quad = 2(Gx - y)^T G \quad [\text{row vector}]$
- Gradient = transpose of derivative = $2 G^T (Gx - y) [\text{column vector}]$

Matrix Derivatives

- What happens when there are **complex variables** involved ?

- Example: Let $w := x + iy$ Let $f(w) := |w|^2 = w^* w = x^2 + y^2$

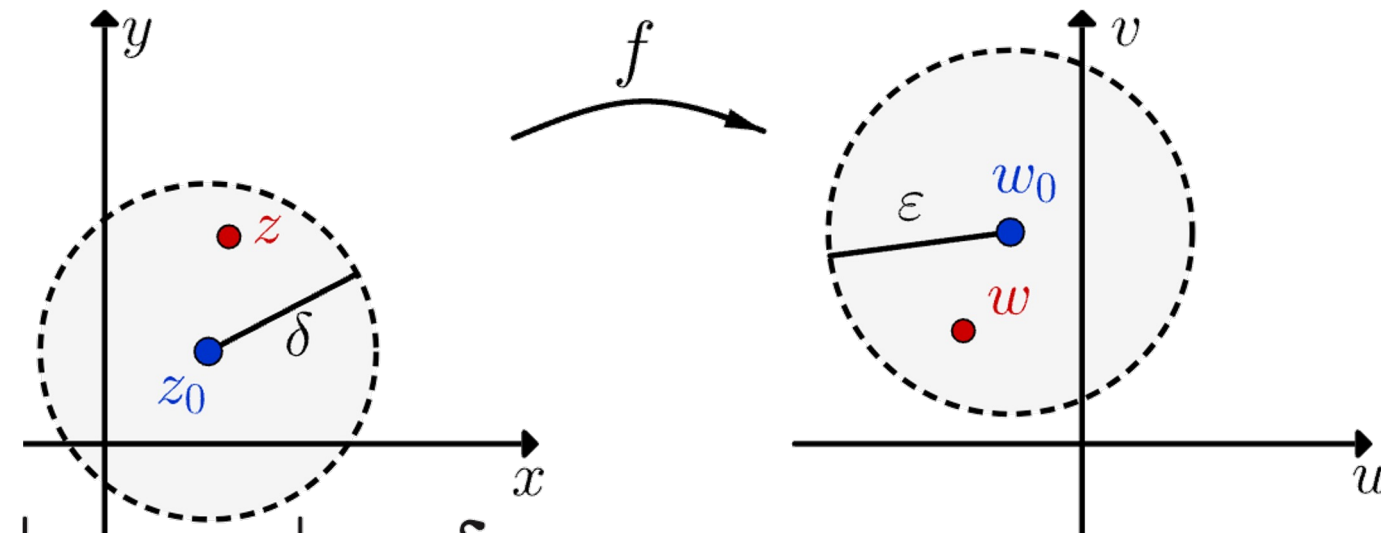
- Complex-derivative definition: $f'(z) := \lim_{h \rightarrow 0} \frac{f(z+h) - f(z)}{h}$ where $h \in \mathbb{C}$

- Complex-limit definition:

$$\lim_{z \rightarrow z_0} f(z) = w_0$$

- For every (sufficiently small) $\epsilon > 0$, there is a $\delta > 0$, such that

$$|f(z) - w_0| < \epsilon \quad \text{whenever} \quad 0 < |z - z_0| < \delta$$



- Complex derivatives differ A LOT from derivatives w.r.t. real variables:
 - because h allowed to vary across all complex (not just real) numbers,
 - and then “**i**” comes into picture

Matrix Derivatives

- Complex derivative
- Let a general $f(x + iy) := u(x, y) + i v(x, y)$
where $u(\cdot)$, $v(\cdot)$ are real-valued functions

1) Take limit by **varying** h along **real** axis ($h := \Delta \in \mathbb{R}$)

$$\begin{aligned} f'(z) &= \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{f(x + \Delta + iy) - f(x + iy)}{\Delta} \\ &= \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{u(x + \Delta, y) + iv(x + \Delta, y) - u(x, y) - iv(x, y)}{\Delta} \\ &= \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{u(x + \Delta, y) - u(x, y)}{\Delta} + i \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{v(x + \Delta, y) - v(x, y)}{\Delta} \\ &= u_x(x, y) + iv_x(x, y) \end{aligned}$$

Matrix Derivatives

- Complex derivative
- Let a general $f(x + iy) := u(x, y) + i v(x, y)$
where $u(\cdot)$, $v(\cdot)$ are real-valued functions

2) Take limit by **varying** h along **imag** axis ($h := i\Delta$, $\Delta \in \mathbb{R}$)

$$\begin{aligned} f'(z) &= \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{f(x + i(y + \Delta)) - f(x + iy)}{i\Delta} \\ &= \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{u(x, y + \Delta) + iv(x, y + \Delta) - u(x, y) - iv(x, y)}{i\Delta} \\ &= \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{u(x, y + \Delta) - u(x, y)}{i\Delta} + i \lim_{\Delta \rightarrow 0, \Delta \in \mathbb{R}} \frac{v(x, y + \Delta) - v(x, y)}{i\Delta} \\ &= -iu_y(x, y) + v_y(x, y) \end{aligned}$$

Matrix Derivatives

- Complex derivative
- $(1) = (2) \Rightarrow$ **Cauchy-Riemann equations**
must be satisfied: $u_x = v_y$ and $u_y = -v_x$
 - Necessary conditions, and sufficient too
- A complex-valued function induces a vector-valued real function
- Given: $f(x + iy) := u(x,y) + i v(x,y)$
where $u(x,y)$, $v(x,y)$ are real-valued functions
- Then, $f(x+iy)$ induces $f(x,y) := (u(x,y), v(x,y))$
- Cauchy-Riemann equations imply that
 2×2 Jacobian matrix of $f(x,y)$ has a special symmetry
- **Any non-zero scalar-valued function of form $(u(x,y), 0)$
doesn't have a complex derivative**

Matrix Derivatives

- What happens when there are complex variables involved ?
- Example: Let $w := x + iy$ Let $f(w) := |w|^2 = w^* w = x^2 + y^2$
- How to perform gradient descent on $f(w)$, to update w ? $\partial/\partial w f(w) = ?$
- Take “real derivative” of $w^* w$
 - Works for a real-valued function of a complex variable
- Key idea: Analogy between complex domain and some larger-dimensional real domain, e.g., $w = x + iy \rightarrow [x, y]^T$
- Then, $\partial/\partial x (x^2 + y^2) = 2x$, $\partial/\partial y (x^2 + y^2) = 2y$
- Thus, $\partial/\partial [x, y]^T (x^2 + y^2) = [2x, 2y] = 2[x, y]$
- If w is a complex-valued vector $[w_1, \dots, w_n]^T$ and if $f(w) = w^{*T} w$, then $\nabla_w f(w) = [2w_1, \dots, 2w_n]^T = 2 w$

Matrix Derivatives

The Complex Gradient Operator and the $\mathbb{C}\mathbb{R}$ -Calculus

June 25, 2009

- For scalar-valued functions of complex variables

- arxiv.org/pdf/0906.4835 $f(z) = f(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}$

Ken Kreutz-Delgado

- “trick is to exploit the real $\mathbb{R}^2$ vector space structure which underlies $\mathbb{C}$ when performing gradient-based optimization.”

Electrical and Computer Engineering
Jacobs School of Engineering
University of California, San Diego

- Using Matrix Calculus

- Get derivatives www.ee.ic.ac.uk/hp/staff/dmb/matrix/special.html#Complex

- Complex-to-real isomorphism (their “trick”)

- “We can associate a complex matrix $C[m \times n]$ with a corresponding real matrix $R[2m \times 2n]$

by replacing each complex element, z , of C

by a 2×2 real matrix $\begin{bmatrix} \operatorname{Re}(z) & -\operatorname{Im}(z) \\ \operatorname{Im}(z) & \operatorname{Re}(z) \end{bmatrix} = |z| \times \begin{bmatrix} \cos(t) & -\sin(t) \\ \sin(t) & \cos(t) \end{bmatrix}$

where $t = \arg(z)$.”

- Gradient = conjugate transpose of derivative

Augustin-Louis Cauchy

- 1789-1857
- Single-handedly founded complex analysis
- More concepts and theorems have been named for Cauchy than for any other mathematician
 - In elasticity alone, 16 concepts, theorems
- Prolific writer
 - 800 research articles, 5 textbooks
 - Number of papers second only to Euler
- Lagrange was a family friend
- Grew up in home of a staunch royalist
 - Family had to flee during revolution



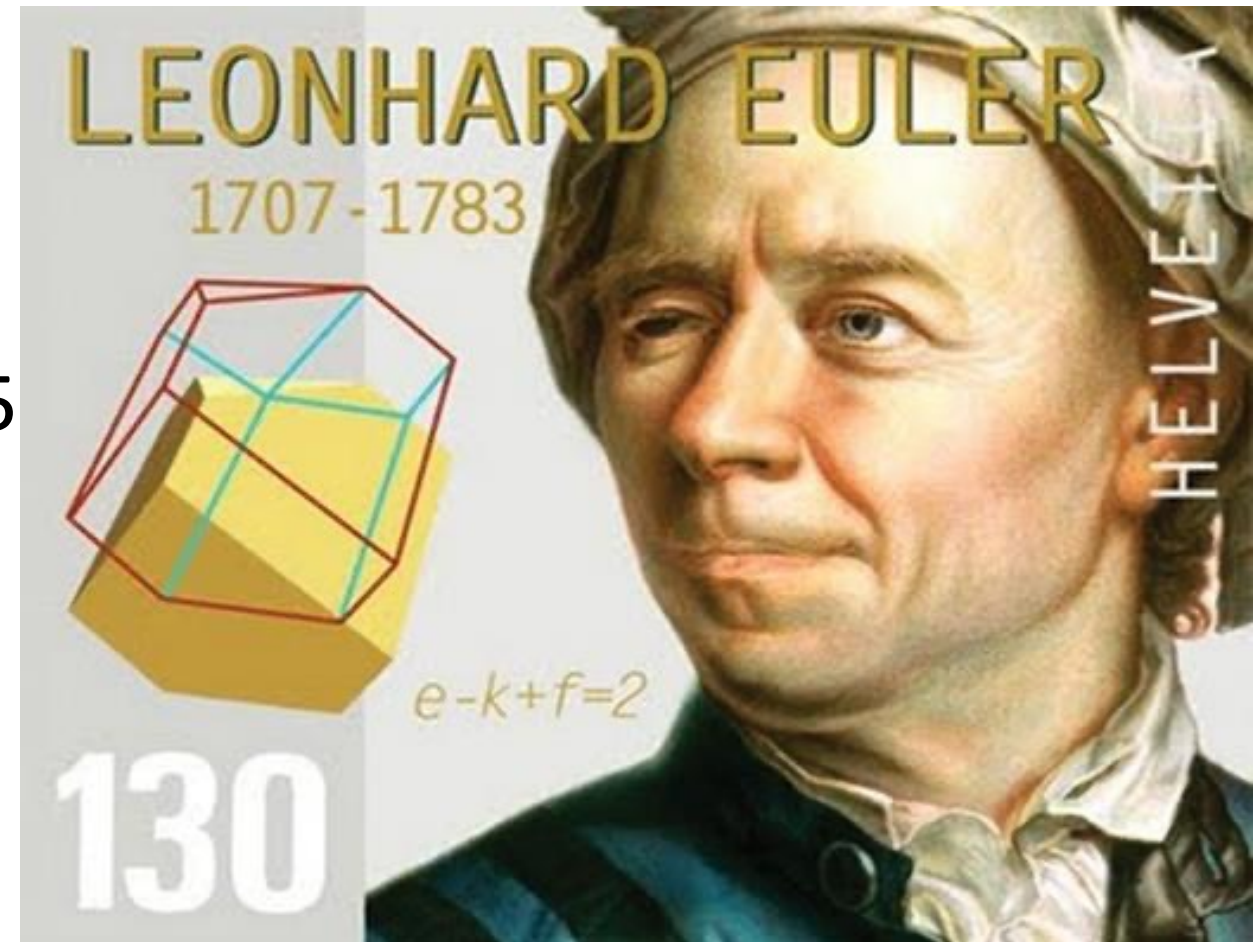
Leonhard Euler

- Mathematician, physicist, astronomer, logician, engineer, 1707-1783
- Laplace said: "Read Euler, read Euler, he is the master of us all."
- Advisor: Bernoulli
- Students: Lagrange
 - Lagrange's students → Fourier, Poisson
- Age 13 → entered university
- Age 19 → got PhD
- Born in Basel. Friend of Bernoulli family
- 866 publications > anybody in his field



Leonhard Euler

- Age 13, in Univ, was studying philosophy to become pastor, like his father
 - Math skills noted by Johann Bernoulli ← then Europe's foremost mathematician
 - Bernoulli convinced Euler's father to let Euler study math
- Lost eyesight due to cataract in 1766
 - "Now I will have fewer distractions"
 - Productivity increased, using scribes
 - On average, 1 math paper per week in 1775
- Only mathematician to have 2 constants named
 - Euler's number = $e = 2.71828...$
 - Euler–Mascheroni constant = $0.57721...$



Gradient Descent

- Cauchy suggested it in 1847
 - Hadamard (mathematician) independently proposed a similar method in 1907
 - Studied extensively by Haskell Curry (computer scientist) in 1944
- Relates to “method of steepest descent”
 1. Set iteration counter $k = 0$ and make an initial guess $\mathbf{x}_0$ for the minimum.
Pick ϵ a tolerance.
 2. Loop:
 1. Compute a descent direction $\mathbf{p}_k$.
 2. Define a one-dimensional function $h(\alpha_k) = f(\mathbf{x}_k + \alpha_k \mathbf{p}_k)$, representing the function value on the descent direction given the step-size.
 3. Find an α_k that minimizes h over $\alpha_k \in \mathbb{R}_+$.
 4. Update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$, and $k = k + 1$
 3. Until $\|\nabla f(\mathbf{x}_{k+1})\| < \epsilon$

Gradient Descent

- Line search: en.wikipedia.org/wiki/Line_search
- Can be done over an interval (or not)
- Many methods exist:
 - Some rely on values of function only (without using function derivatives)
 - [Golden-section search](#)
 - Some rely on values of function and (directional) derivatives
 - [Newton's method](#)
 - [Quasi-Newton methods](#)
- But line-search can be expensive.
It can be avoided,
if we can tolerate intermediate sub-optimal solution
(as long as we improve over current estimate)

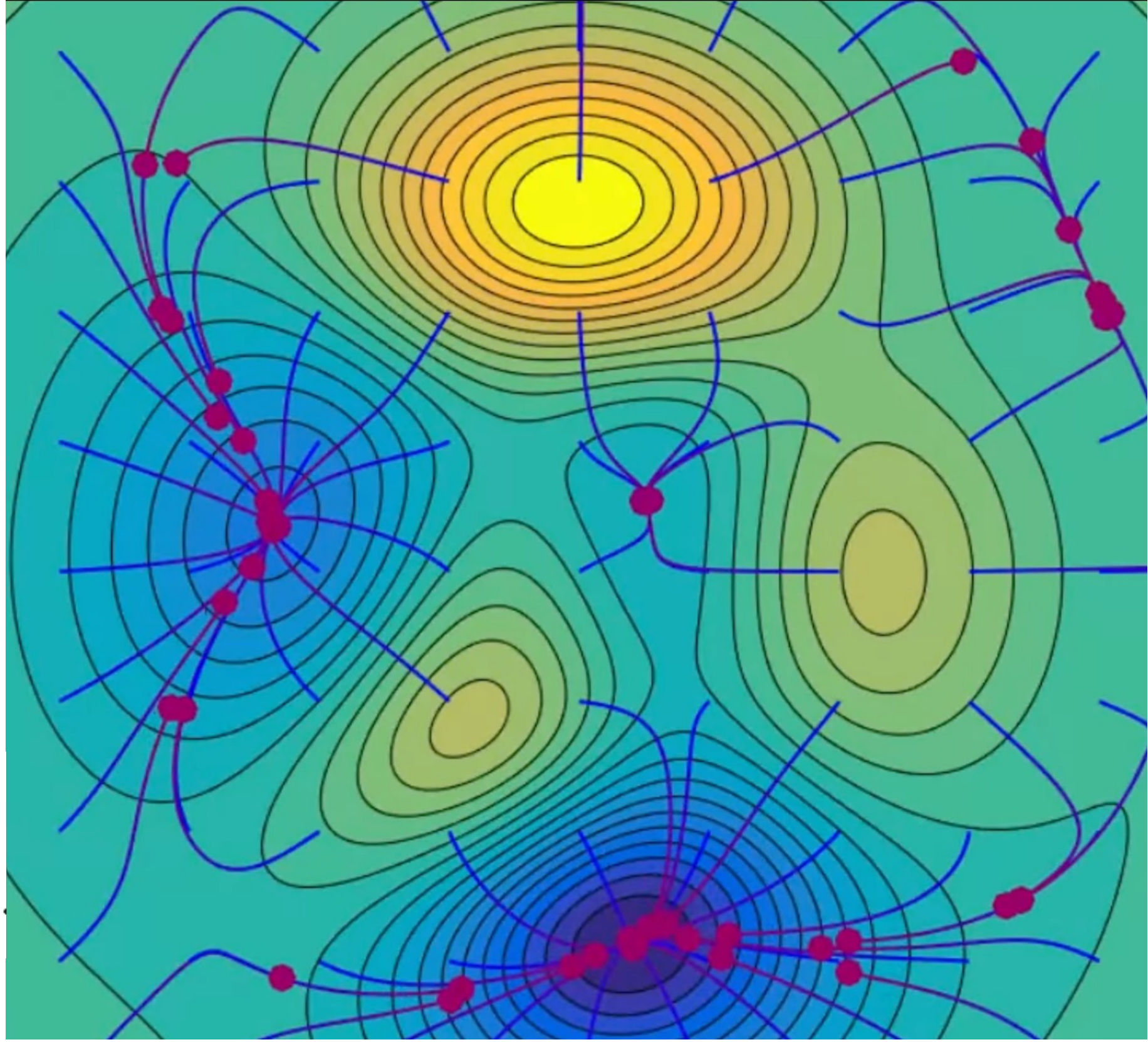
Gradient Descent

- Gradients and Contours: what is the relationship ?
- In infinitesimally-small neighborhood of $\mathbf{x}$, what is direction along which:
 - $f(\mathbf{x})$ is constant ?
 - $f(\mathbf{x})$ increases fastest ?
 - $f(\mathbf{x})$ decreases fastest ?

- Taylor-series expansion

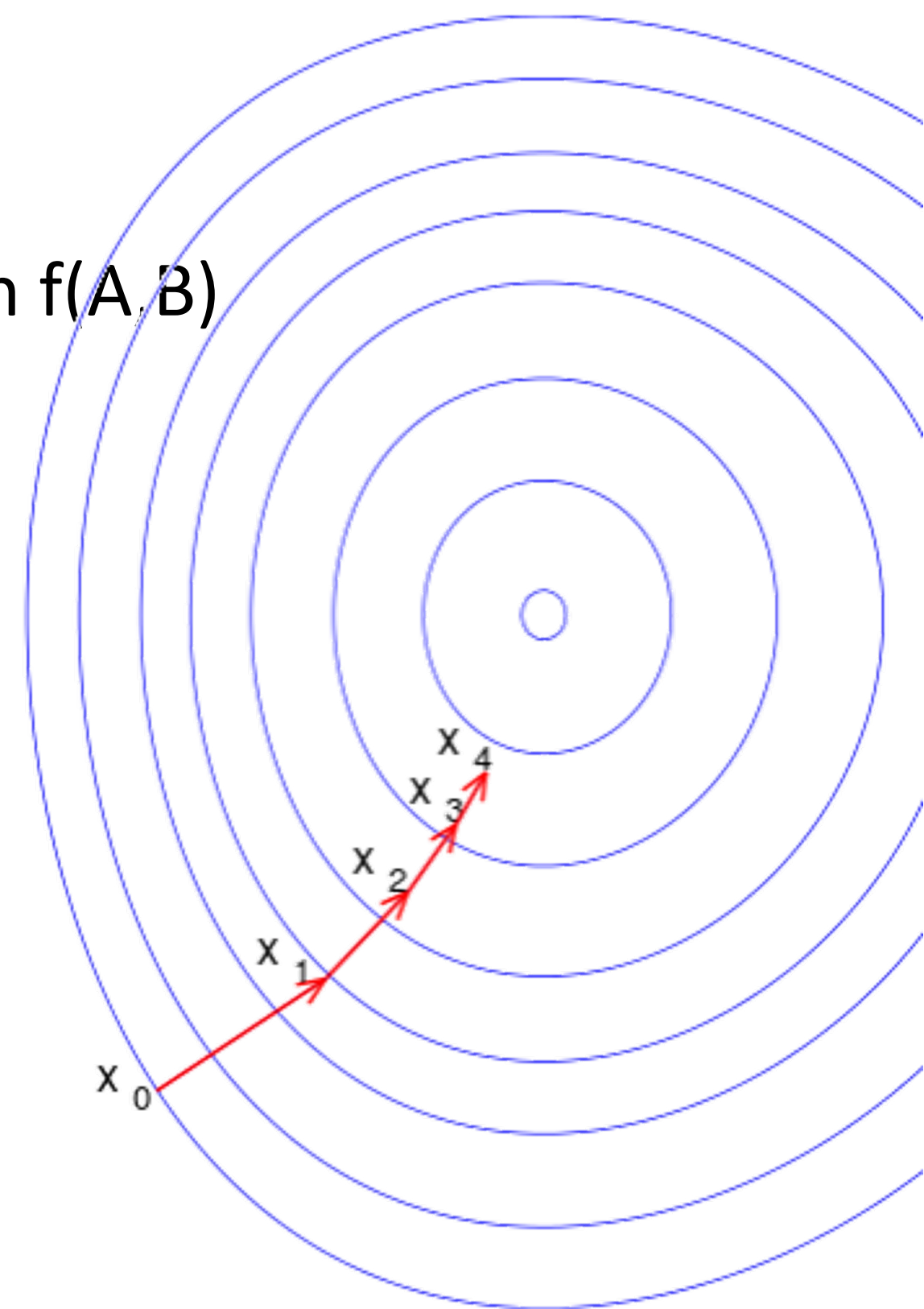
$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + J(\mathbf{x})\Delta\mathbf{x}$$

- Restrict $\Delta\mathbf{x}$ to fixed norm



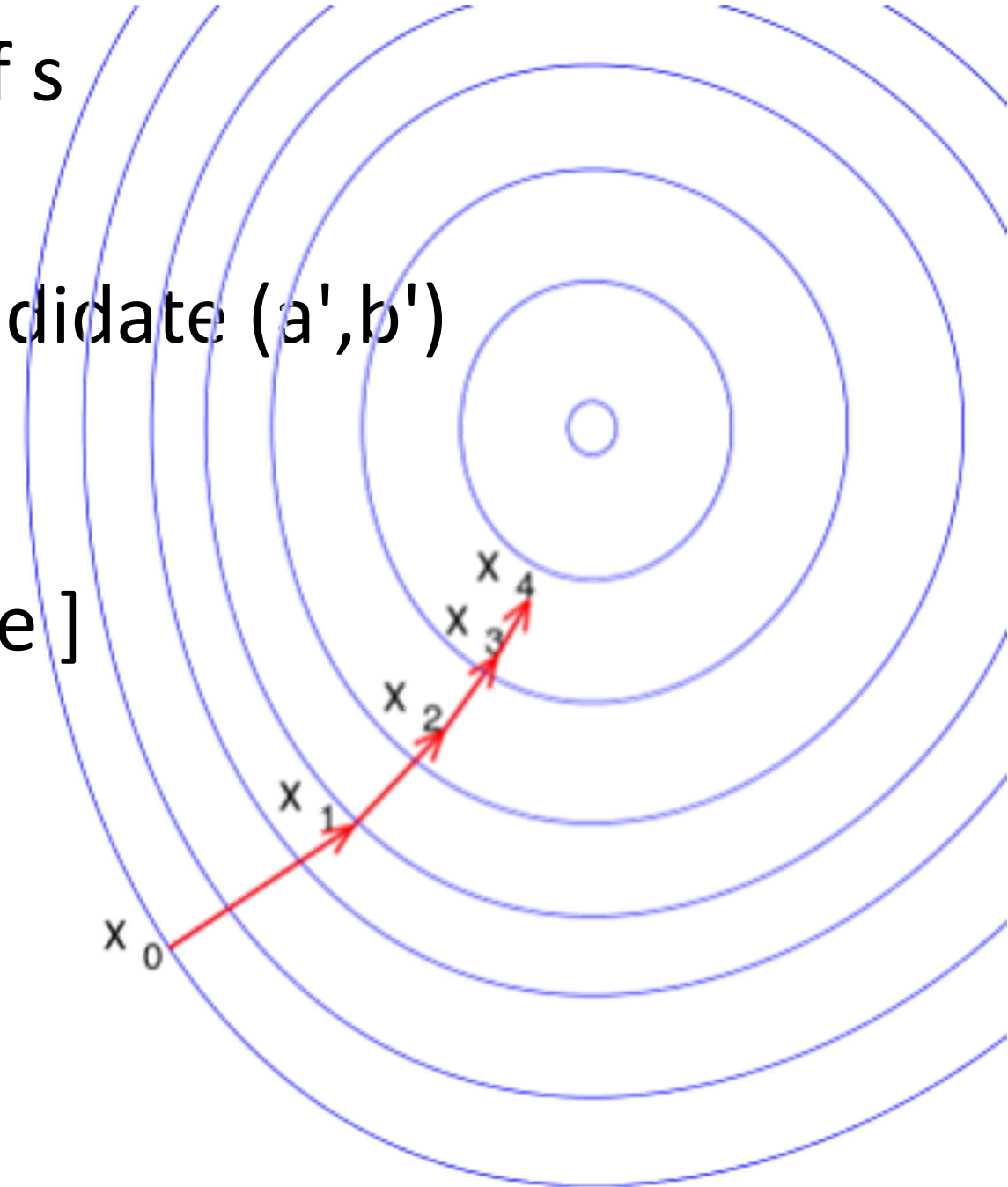
Optimization Methods

- Gradient-descent algorithm
- Blue curves = contours of objective function $f(A, B)$
- Let current estimate = (a, b)
- How to improve estimate ?
- Compute gradient at (a, b) :
 $g(a, b) = (df/dA, df/dB)$
evaluated at $(A=a, B=b)$
- Improved solution $(a', b') = (a, b) - s g(a, b)$
where $s > 0$ is small step size
 - How to find (a', b') and update s ?
- Repeat until improvement is negligible



Optimization Methods

- Gradient-descent algorithm: how to find (a',b') and update s
- Start with some value (or previous value) of s
- Propose candidate $(a',b') = (a,b) - s g(a,b)$
- Compare $f(.)$ values at current (a,b) and candidate (a',b')
- If $f(a',b') < f(a,b)$,
then found (a',b') , update $s \leftarrow s * 1.1$
[increase s for use in future gradient update]
- If $f(a',b') \geq f(a,b)$,
then $s \leftarrow s/2$ [reduce s]
- Repeat ProposeCandidate+Check
until improved solution (a',b') found
OR s tends to zero



Optimization Methods

- To care or not to care about elusive global optimum !
- What is a bigger problem ?
 - 1) Optimization algorithm getting stuck in a local optimum ?
OR
 - 2) Designing a statistical model (e.g., prior, likelihood, etc.) and an optimization problem (e.g., max-a-posteriori, median, mean, etc.) whose global optimum isn't same as ground truth ?
OR
 - 3) Designing a model-learning formulation where model corresponding to global optimum overfits to training data (generalizes poorly to test data) ?

Bad Global Minima Exist and SGD Can Reach Them

Shengchao Liu

Quebec Artificial Intelligence Institute (Mila)

Université de Montréal

liusheng@mila.quebec

Dimitris Papailiopoulos

University of Wisconsin-Madison

dimitris@papail.io

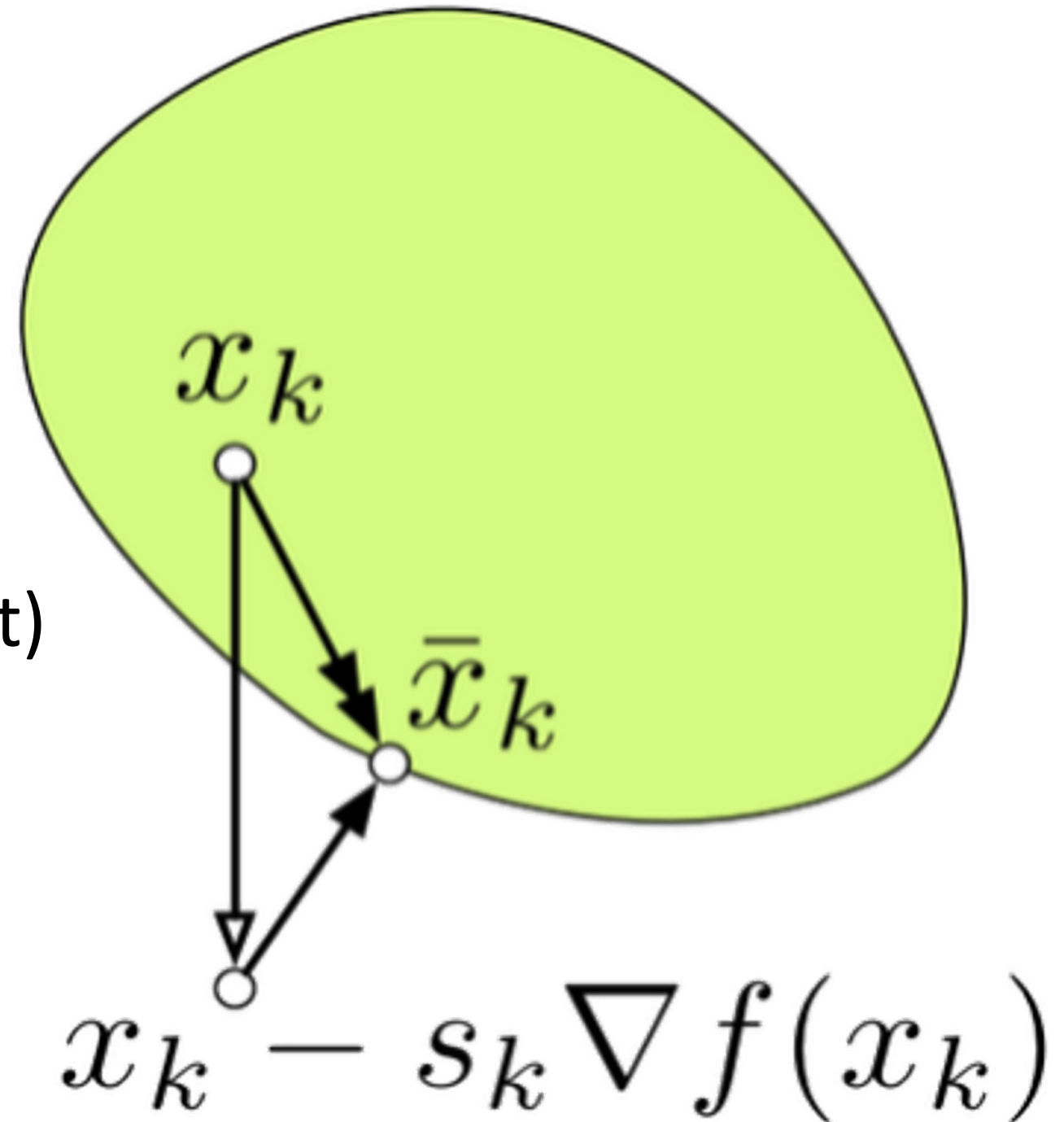
Dimitris Achlioptas

University of Athens

optas@di.uoa.gr

Optimization Methods

- **Projected gradient-descent**
- Algorithm:
 - (1) Take a step along negative gradient
 - (2) Project back onto constraint set
- For convex constraint sets, behaves similar to gradient descent w.r.t. convergence (to a stationary point)
- When does this converge ?
 - 1) When gradient is zero
 - OR
 - 2) When ...



Optimization Methods

- Newton's method
- Scalar-valued function $f(\cdot)$ of **single-variable** x
- Taylor series expansion (up to 2nd-order terms)

$$f_T(x_n + \Delta x) = f(x_n) + f'(x_n)\Delta x + \frac{1}{2}f''(x_n)\Delta x^2,$$

- Minimize $f(x_n + \Delta x)$ w.r.t. Δx
- Take derivative w.r.t. Δx , equate it to 0

$$f'(x_n) + f''(x_n)\Delta x = 0$$

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

Optimization Methods

- Newton's method
- **Scalar-valued** function $f(\cdot)$ of **multi-variable** $\mathbf{x}$
- Taylor series expansion (up to 2nd-order terms)

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + J(\mathbf{x})\Delta\mathbf{x} + \frac{1}{2}\Delta\mathbf{x}^T H(\mathbf{x})\Delta\mathbf{x}$$

- J := Jacobian **vector** of $f(\cdot)$
- H := Hessian **matrix** of $f(\cdot)$:= transpose (Jacobian (gradient ($f(\cdot)$)))
 - If second partial derivatives are all continuous, then Hessian is a symmetric matrix (and transpose doesn't matter)

$$(\mathbf{H}_f)_{i,j} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

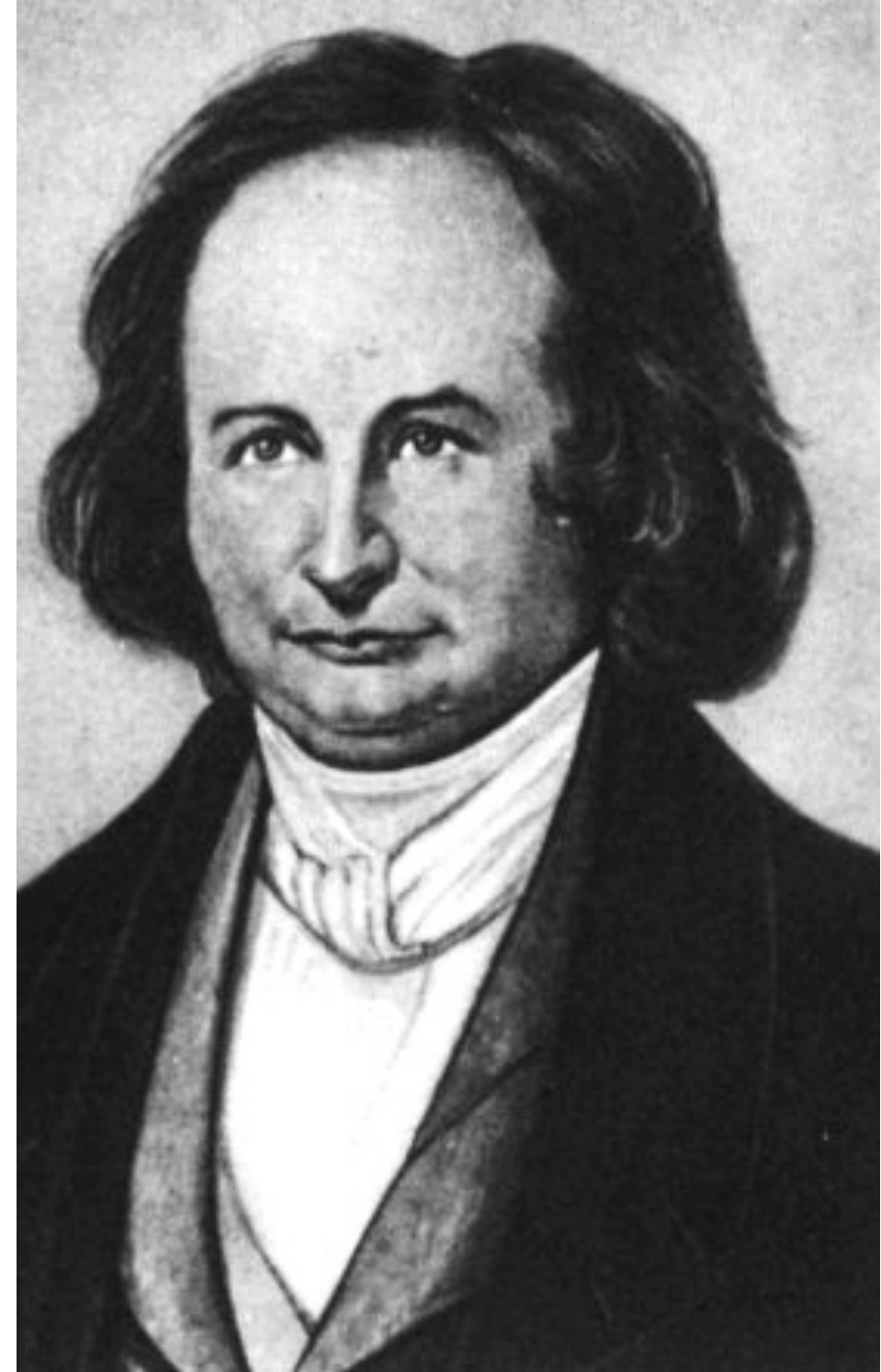
Optimization Methods

- **Jacobian**
- $F: \mathbb{R}^n \rightarrow \mathbb{R}^m$
- m-valued function of n variables

$$\mathbf{J}_f = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^\top f_1 \\ \vdots \\ \nabla^\top f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Carl Gustav Jacob Jacobi

- Mathematician
- Prodigy, home schooled by his uncle
- Age 12, joins top secondary school in Germany
- In 6 months, moved to senior year (~ grade 12)
- Remained in same class until 16 (when university accepted him)
- Age 21, got PhD (Univ. Berlin)
- Age 25, full prof at Konisberg Univ.
- Age 38, breakdown from overwork
 - Visits Italy to relax, back to Berlin, retires, pension
- Age 47, smallpox, death



Optimization Methods

- **Hessian**
- Defined only for real-valued functions $f(\cdot)$ of n -variables

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Otto Hesse

- Mathematician
- Advisor: Jacobi
- Students
 - Gustav Kirchhoff
 - Kirchhoff students
 - Max Planck
→ Nobel prize in Physics, 1918
 - Dmitri Mendeleev
→ Periodic table



Optimization Methods

- Newton's method
- Scalar-valued function $f(\cdot)$ of multi-variable $\mathbf{x}$
- Taylor series expansion (up to 2nd-order terms)

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + J(\mathbf{x})\Delta\mathbf{x} + \frac{1}{2}\Delta\mathbf{x}^T H(\mathbf{x})\Delta\mathbf{x}$$

- Minimize $f(\mathbf{x} + \Delta\mathbf{x})$ w.r.t. $\Delta\mathbf{x}$
- Take derivative w.r.t. $\Delta\mathbf{x}$, and equate it to 0
- $0 = J(\mathbf{x}) + 0.5 \Delta\mathbf{x}^T (H(\mathbf{x}) + H(\mathbf{x})^T)$
- $0 = J(\mathbf{x})^T + H(\mathbf{x}) \Delta\mathbf{x}$

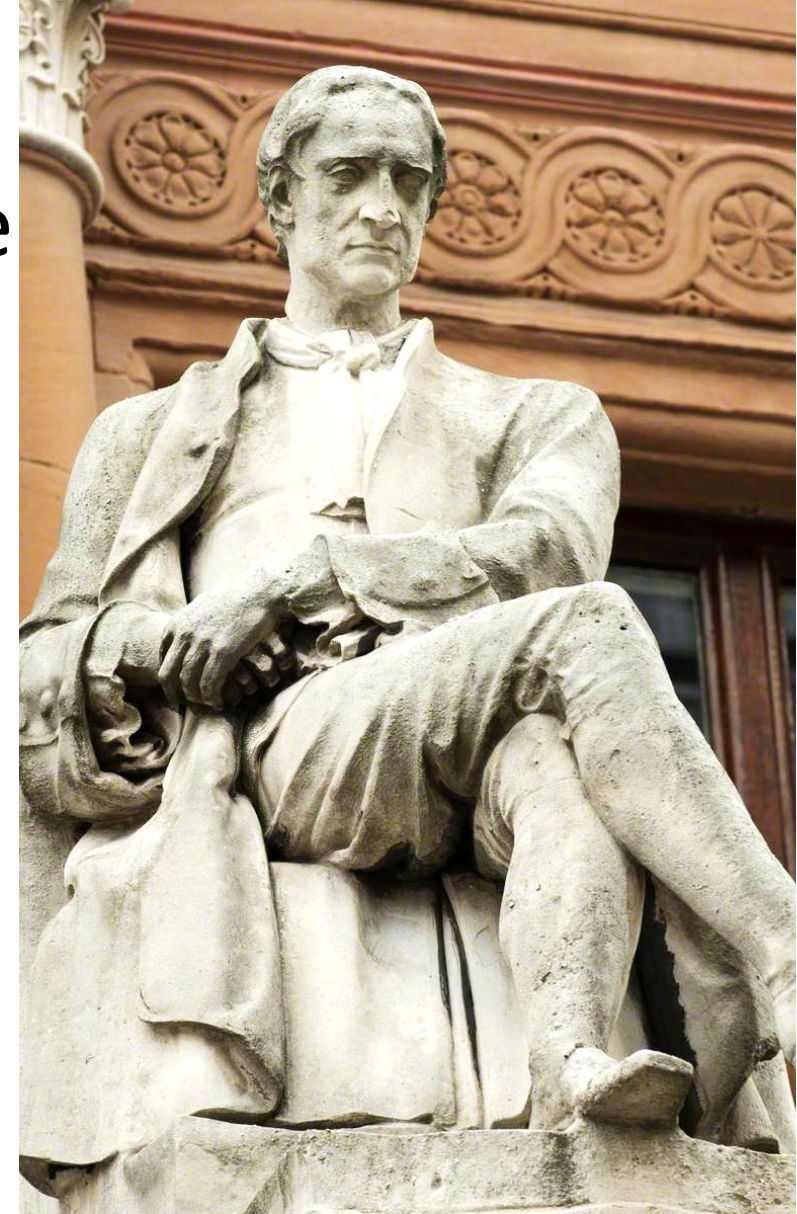
Sir Isaac Newton

- Mathematician, physicist, astronomer, 1642 - 1726
- His epitaph, suggested by Alexander Pope,
 - Nature and nature's laws lay hid in night;
God said "Let Newton be" and all was light.
- Newton wrote in his memoir
 - I do not know what I may appear to the world,
but to myself,
I seem to have been only like a boy playing on the sea-shore,
and diverting myself, in now and then,
finding a smoother pebble or a prettier shell than ordinary,
whilst the great ocean of truth lay all undiscovered before me.
- In a letter to rival Robert Hooke
 - If I have seen further, it is by standing on the shoulders of giants



Sir Isaac Newton

- Born premature, father died, wasn't expected to live
- In school, loved chemistry.
Mother pulled him out of school.
Planned to make him a farmer.
Newton found farming monotonous
- Went to Cambridge Univ., to study, later professor
- Philosophiae Naturalis Principia Mathematica
 - Established classical mechanics
 - Single most influential book on physics, perhaps science
- As a person
 - Never married or had (m)any friends.
In later years, mentally unstable due to pride, insecurity, alchemy.



Optimization Methods

- Gauss-Newton algorithm
- For non-linear least-squares optimization
- n variables: $\beta := (\beta_1, \dots, \beta_n)^\top$
- Design m-valued function (e.g., m measurements) as $r := (r_1, \dots, r_m)^\top$
where i-th “residual” is $r_i(\beta) = y_i - f(x_i, \beta)$
- Design optimization problem: Find β to minimize $S(\beta) = \sum_{i=1}^m r_i(\beta)^2$
 - Least-squares fit for $y = f(x)$ with nonlinear $f(\cdot)$
- Gradient of real-valued obj. function: j-th component = $g_j = 2 \sum_{i=1}^m r_i \frac{\partial r_i}{\partial \beta_j}$
- Hessian of real-valued obj. function: jk-th component

$$H_{jk} = 2 \sum_{i=1}^m \left(\frac{\partial r_i}{\partial \beta_j} \frac{\partial r_i}{\partial \beta_k} + r_i \frac{\partial^2 r_i}{\partial \beta_j \partial \beta_k} \right)$$

Optimization Methods

- Gauss-Newton algorithm
- n variables: $\beta = (\beta_1, \dots, \beta_n)^\top$
- Residual i: $r_i(\beta) = y_i - f(x_i, \beta)$
- Optimization problem: Find β to minimize $S(\beta) = \sum_{i=1}^m r_i(\beta)^2$
- Hessian of real-valued obj. function $S(\beta)$:
jk-th component
$$H_{jk} = 2 \sum_{i=1}^m \left(\frac{\partial r_i}{\partial \beta_j} \frac{\partial r_i}{\partial \beta_k} + r_i \frac{\partial^2 r_i}{\partial \beta_j \partial \beta_k} \right)$$
 - Assume $\left| r_i \frac{\partial^2 r_i}{\partial \beta_j \partial \beta_k} \right| \ll \left| \frac{\partial r_i}{\partial \beta_j} \frac{\partial r_i}{\partial \beta_k} \right|$
 - Jacobian matrix of m-valued function $r(\beta)$ has ij-th component $J_{ij} = \frac{\partial r_i}{\partial \beta_j}$
 - Then $H_{jk} \approx 2 \sum_{i=1}^m J_{ij} J_{ik}$

Optimization Methods

$$g_j = 2 \sum_{i=1}^m r_i \frac{\partial r_i}{\partial \beta_j} \quad J_{ij} = \frac{\partial r_i}{\partial \beta_j} \quad H_{jk} = 2 \sum_{i=1}^m \left(\frac{\partial r_i}{\partial \beta_j} \frac{\partial r_i}{\partial \beta_k} \right)$$

- Gauss-Newton algorithm

- Apply Newton's method using approximate Hessian

$$\mathbf{g} = 2\mathbf{J}_r^T \mathbf{r}, \quad \mathbf{H} \approx 2\mathbf{J}_r^T \mathbf{J}_r \quad \boldsymbol{\beta}^{(s+1)} = \boldsymbol{\beta}^{(s)} + \Delta; \quad \Delta = -(\mathbf{J}_r^T \mathbf{J}_r)^{-1} \mathbf{J}_r^T \mathbf{r}$$

- Works like Newton's method when:

- Residuals have small magnitude in region near minimum, starting point
- Functions aren't very nonlinear: 2nd (& higher) derivatives small in magnitude

- Advantage over Newton's method

- 2nd derivatives of residuals aren't required

$$\left| r_i \frac{\partial^2 r_i}{\partial \beta_j \partial \beta_k} \right| \ll \left| \frac{\partial r_i}{\partial \beta_j} \frac{\partial r_i}{\partial \beta_k} \right|$$

- In general, convergence NOT guaranteed

- Can diverge
- Introduce a step size $s < 1$ (can tune adaptively) on Δ
- Better way = Levenberg-Marquardt algorithm

Optimization Methods

- Levenberg-Marquardt algorithm (damped least-squares)
- For non-linear least-squares optimization
- Gauss-Newton update (step size s): $\Delta = -s (J^T J)^{-1} J^T r$
- **Modification 1:** $\Delta = -s (J^T J + \lambda I)^{-1} J^T r$, where $\lambda = \text{damping factor} > 0$
 - Larger $\lambda \rightarrow J^T J$ has lesser effect $\rightarrow$ more of gradient-based update
 - Smaller $\lambda \rightarrow$ more of Gauss-Newton update
- **Modification 2:** $\Delta = -s (J^T J + \lambda \text{diag}(J^T J))^{-1} J^T r$
 - Diagonal of $J^T J \sim$ diagonal of Hessian of real-valued obj. function
= 2nd derivatives of real-valued obj. function
= curvature of obj. function in each variable's dimension
 - When λ is large enough ($\gg 0$), as curvatures w.r.t. variables increase, then scaling factors for partial derivatives get smaller